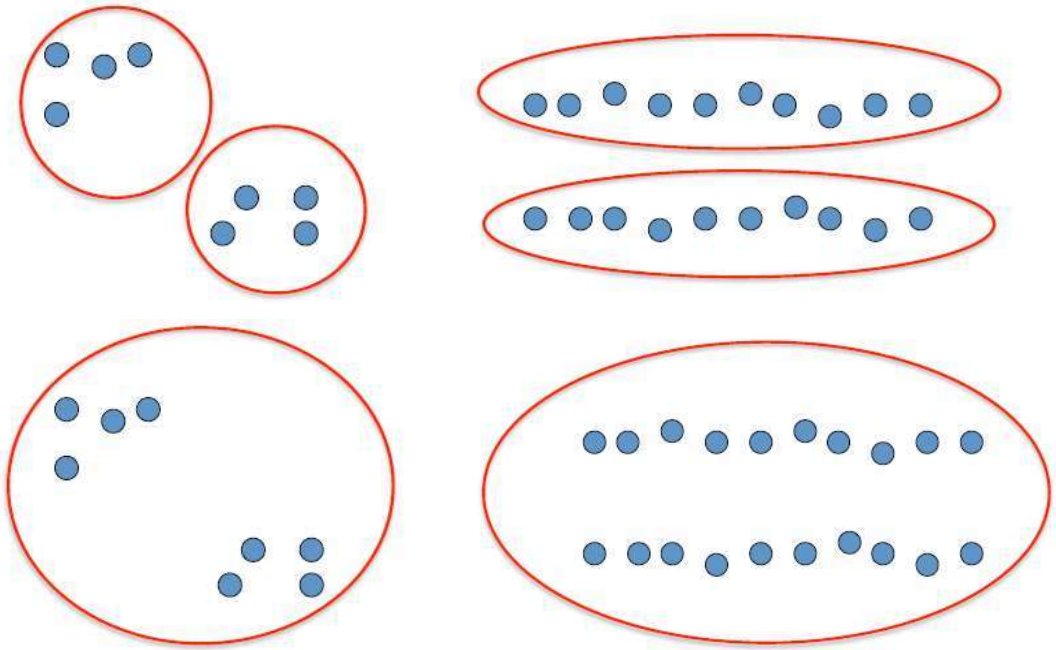
## Clustering:

Clustering results are dependent on the measure of similarity (or distance) between "points" to be clustered

Basic idea: group together similar instances

Example: 2D point patterns



**Clustering**

The classification of objects into different groups.

The partitioning of a data set into subsets (clusters), so that the data in each subset (ideally) share some common trait - often according to some defined distance measure.

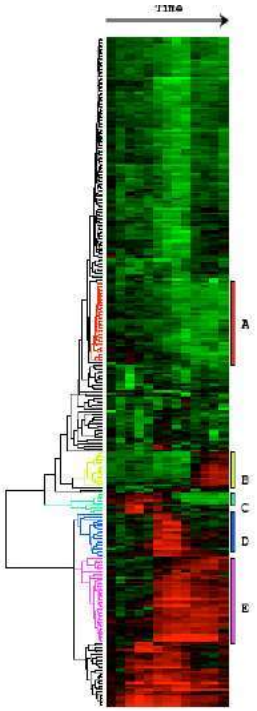
$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Euclidean distance

# Clustering Examples:

- Document/Image/Webpage Clustering
- Image Segmentation (clustering pixels)

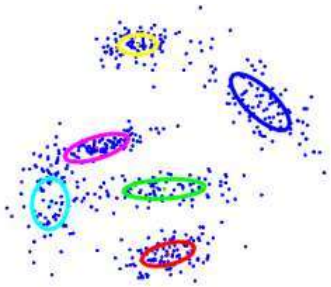
- Clustering web-search results
- Clustering (people) nodes in (social) networks/graphs
- .. and many more..



Clustering expression gene data
Eisen et al, PNAS 1998<

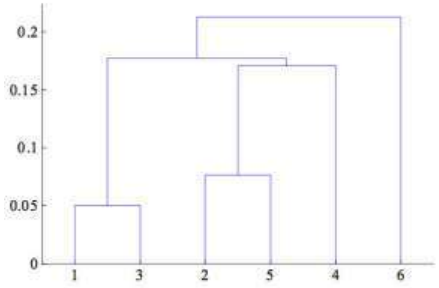
# Types of Clustering:

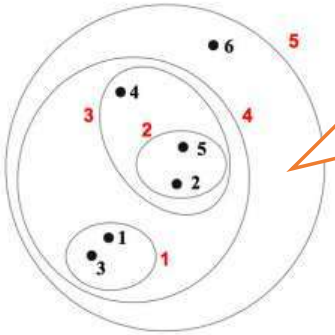
- Flat or Partitional clustering
  - Partitions are independent of each other



- Hierarchical clustering
  - Partitions can be visualized using a tree structure (a dendrogram)

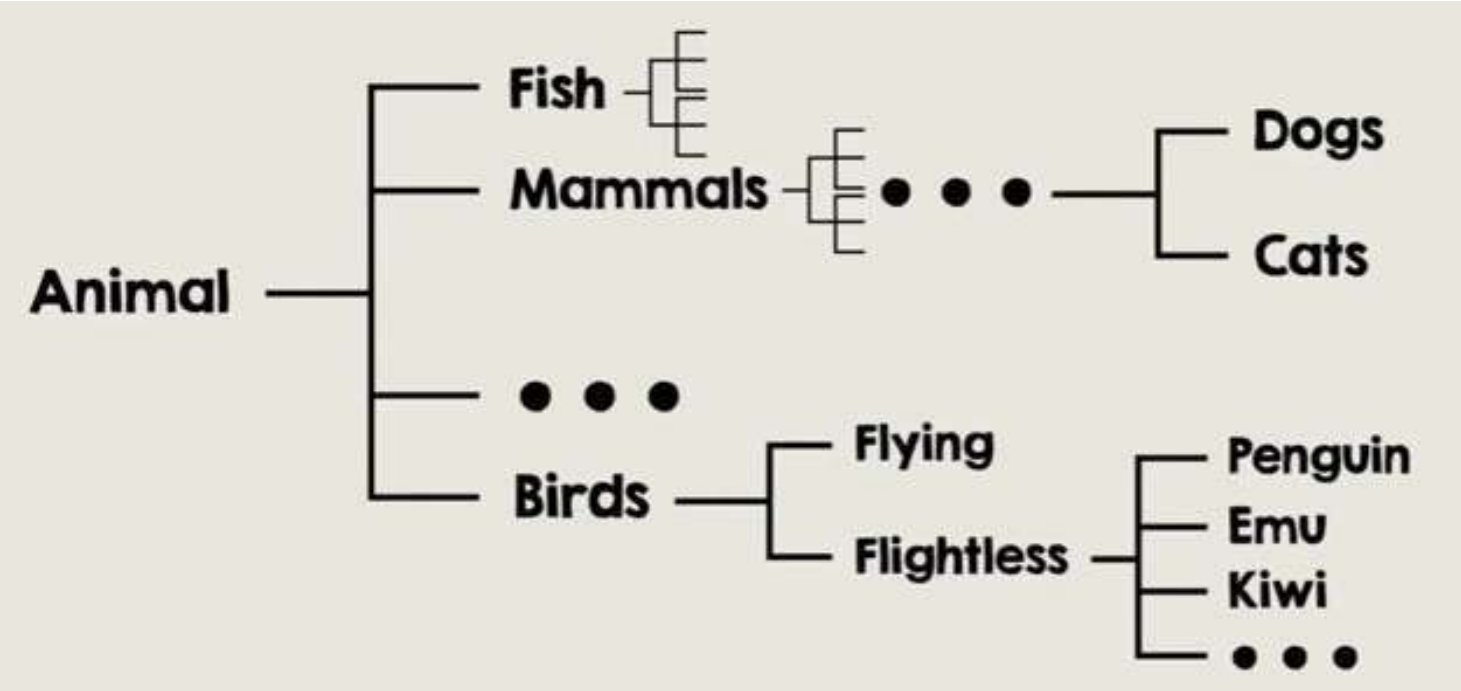
In hierarchical clustering, we can look at a clustering for any given number of cluster by "cutting the dendrogram at an appropriate level (so $K$ does not have to be specified)

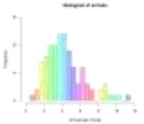



Hierarchical clustering gives us a clustering at multiple levels of granularity

# Hierarchical Clustering :Taxonomy of the Animal Kingdom

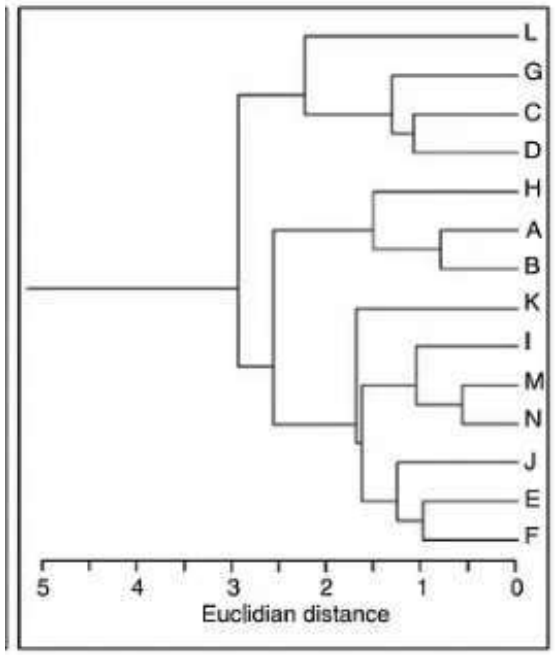


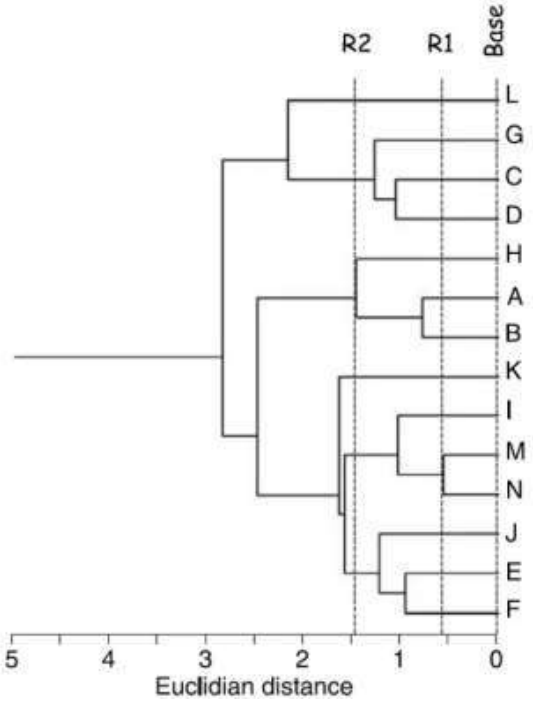
# Hierarchical Clustering

**A *dendrogram* is a diagram used to depict the hierarchical relationship between things.**

we assign each object (data point) to a separate cluster. Then compute the distance (similarity) between each of the clusters




## Using Reference Lines in a Dendrogram:




the vertical line that connects the clusters in a group to be a reference line that extends down to the horizontal scale.

One can determine how similar its children clusters are by seeing where the reference line intersects the horizontal scale.

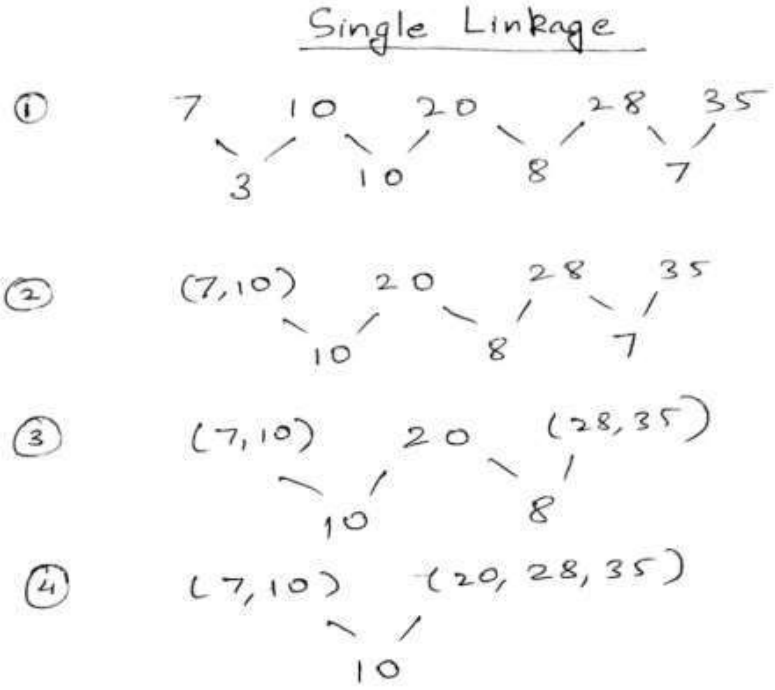
The group connected at R1, M to N, are more similar to each other.

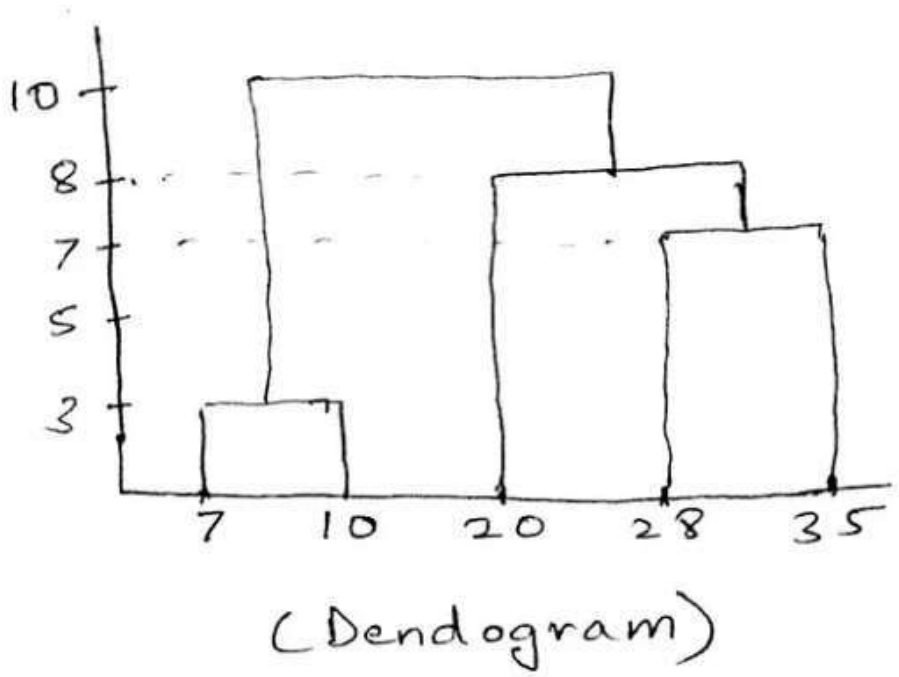
R2 line H to clusters A and B.

# Single Linkage:

For the one dimensional data set **{7,10,20,28,35}**, perform hierarchical clustering and plot the dendogram to visualize it.

In single link hierarchical clustering, we merge in each step the two clusters, whose two closest members have the smallest distance.

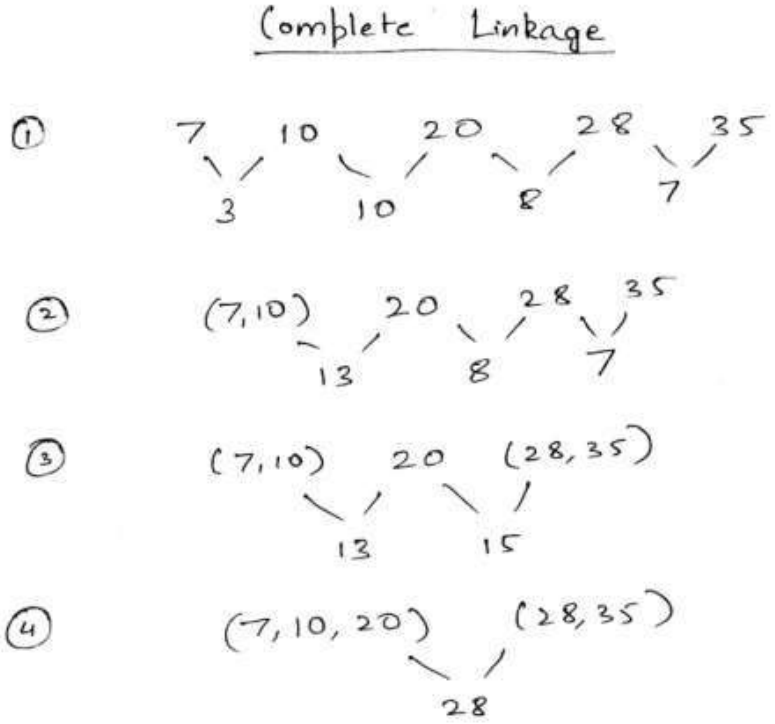


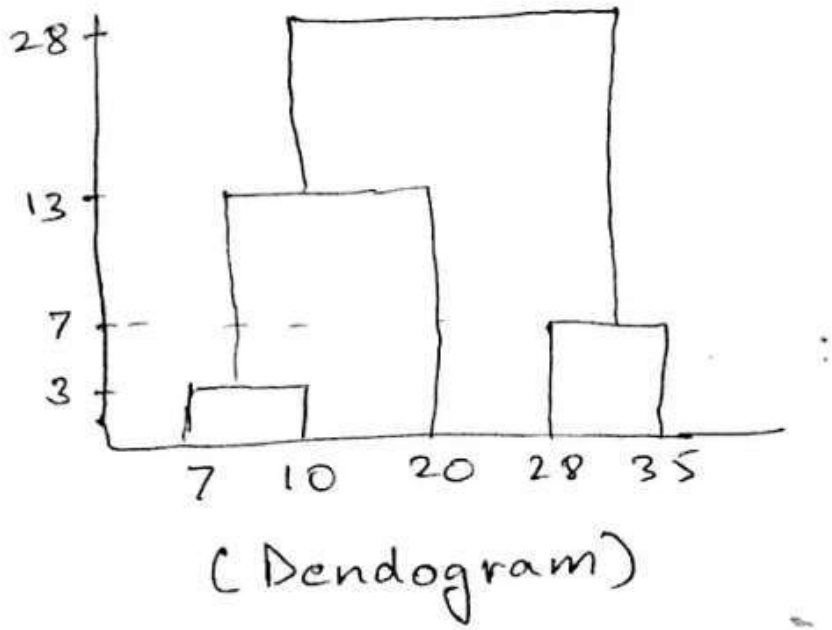



# Complete Linkage:

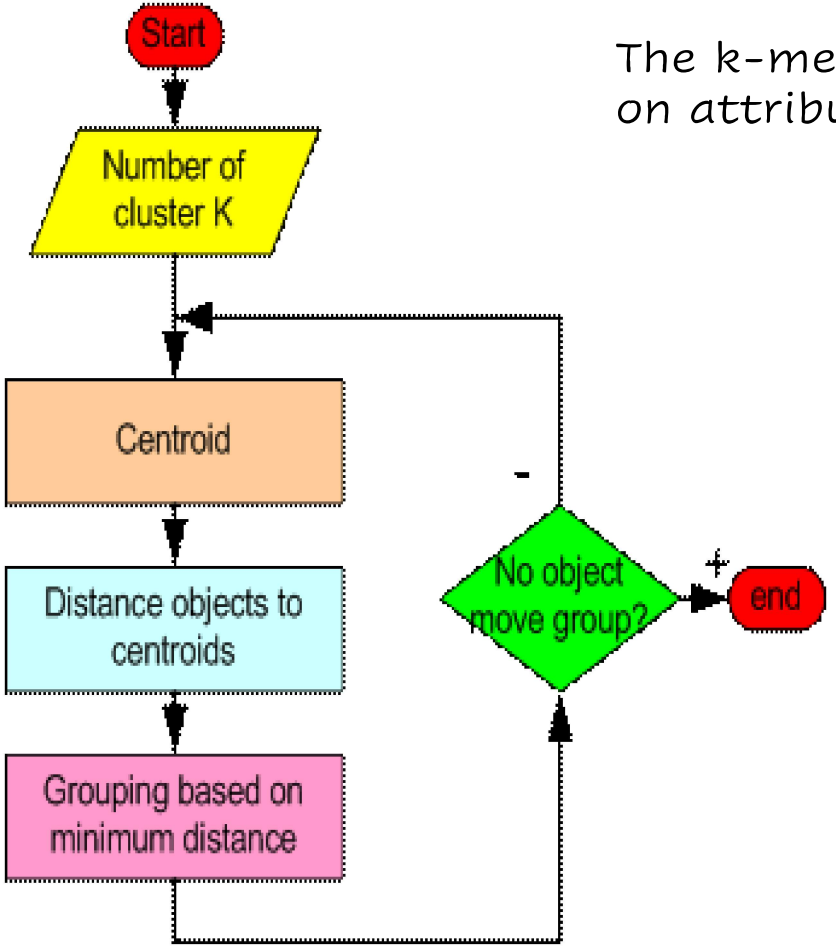
For the one dimensional data set **{7,10,20,28,35}**, perform hierarchical clustering and plot the dendogram to visualize it.

In complete link hierarchical clustering, we merge in the members of the clusters in each step and also the smallest maximum pairwise distance.

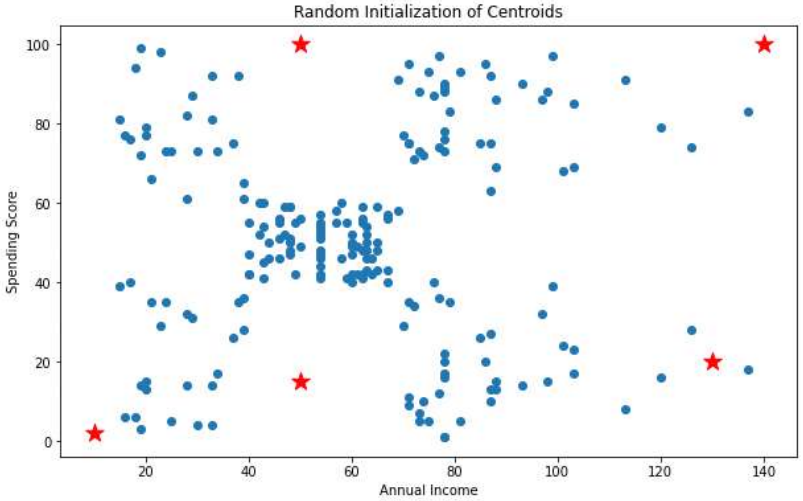


# K-Means FlowChart:




The k-means algorithm is to cluster n objects based on attributes into k partitions, where k < n.



# K-Means Algorithm: Convergence

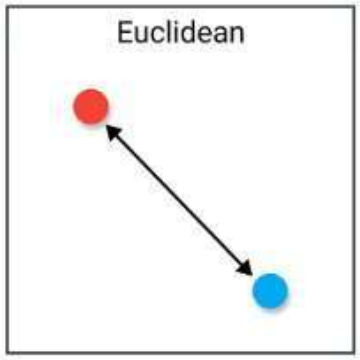
- Cluster means don't change.
- Cluster "Loss" doesn't change much

Some ways to declare convergence if between two successive iterations:

© Stack Overflow Blog

# Euclidean Distance

The Euclidean distance can also be referred to as a L2-norm



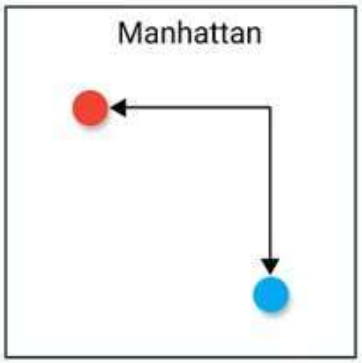

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

```
from scipy.spatial import distance
distance.euclidean(vector_1, vector_2)
```

# Manhattan Distance

The Manhattan distance is also called the Taxicab or City-Block distance as the distance between two real-valued vectors is calculated. It move at right angles.

used for discrete and binary attributes to get a realistic path.




The Manhattan distance is based on a L1-norm and is calculated by:

$$d = \sum_{i=1}^{n} (x_i - y_i)$$

```
from scipy.spatial import distance
distance.cityblock(vector_1, vector_2)
```

*Disadvantages*

It is less intuitive than the Euclidean distance in high dimensional space

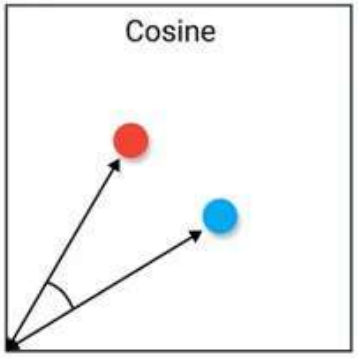
it does not show the shortest path possible.

# Cosine Distance

The Cosine Similarity is a measure of orientation between two vectors (magnitude of the vectors is neglected).

It is used in higher dimensionality where the magnitude of the data does not matter much,

e.g., for recommendation systems or text analyses in which the data is represented by the word count.




$$d = \cos(\alpha) = \frac{x\,y}{\|x\|\,\|y\|}$$

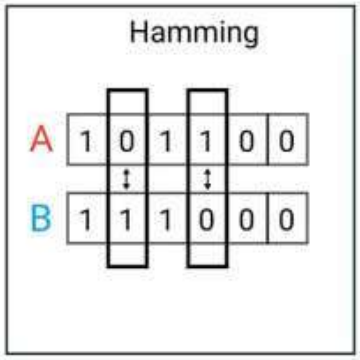
```
from scipy.spatial import distance
distance.cosine(vector_1, vector_2)
```

# Hamming Distance

error detection or error correction when data is transmitted over **computer networks**.

It is also using in coding theory for comparing equal length data words.

The Hamming distance measures the dissimilarity between two binary vectors or strings.




The vectors are compared element-wise and the number of differences. The resulting distance lies between 0 if both vectors are identical and 1 if both vectors are completely different.

```
from scipy.spatial import distance
distance.hamming(vector_1, vector_2)
```

*Disadvantages*

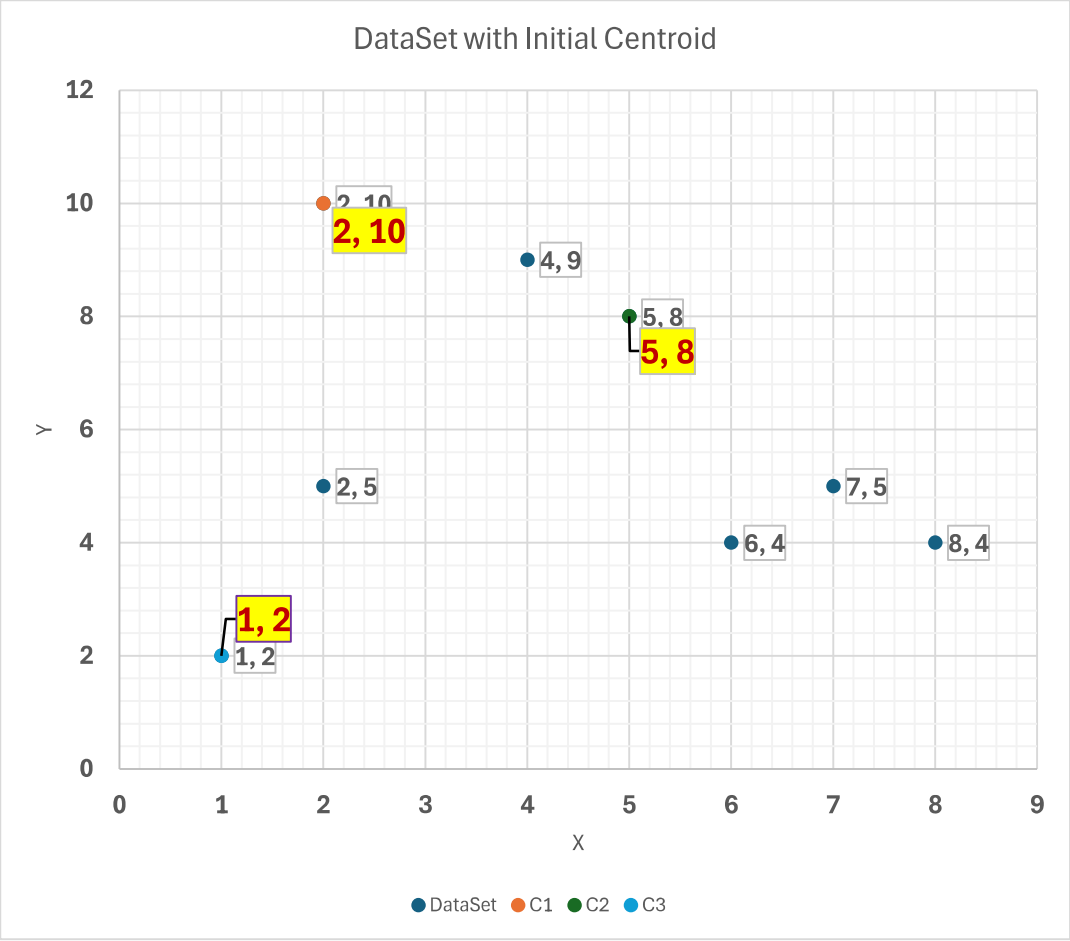
the distance measure can only compare vector of the same length

it does not give the magnitude of the difference.

Hamming distance is not recommended when the magnitude of the difference matters

# K-Mean: Dataset and Plotting

| | Data Set | |
|---|---|---|
| | 2 | 10 |
| | 2 | 5 |
| | 8 | 4 |
| | 5 | 8 |
| | 7 | 5 |
| | 6 | 4 |
| | 1 | 2 |
| | 4 | 9 |
| C1 | 2 | 10 |
| C2 | 5 | 8 |
| C3 | 1 | 2 |



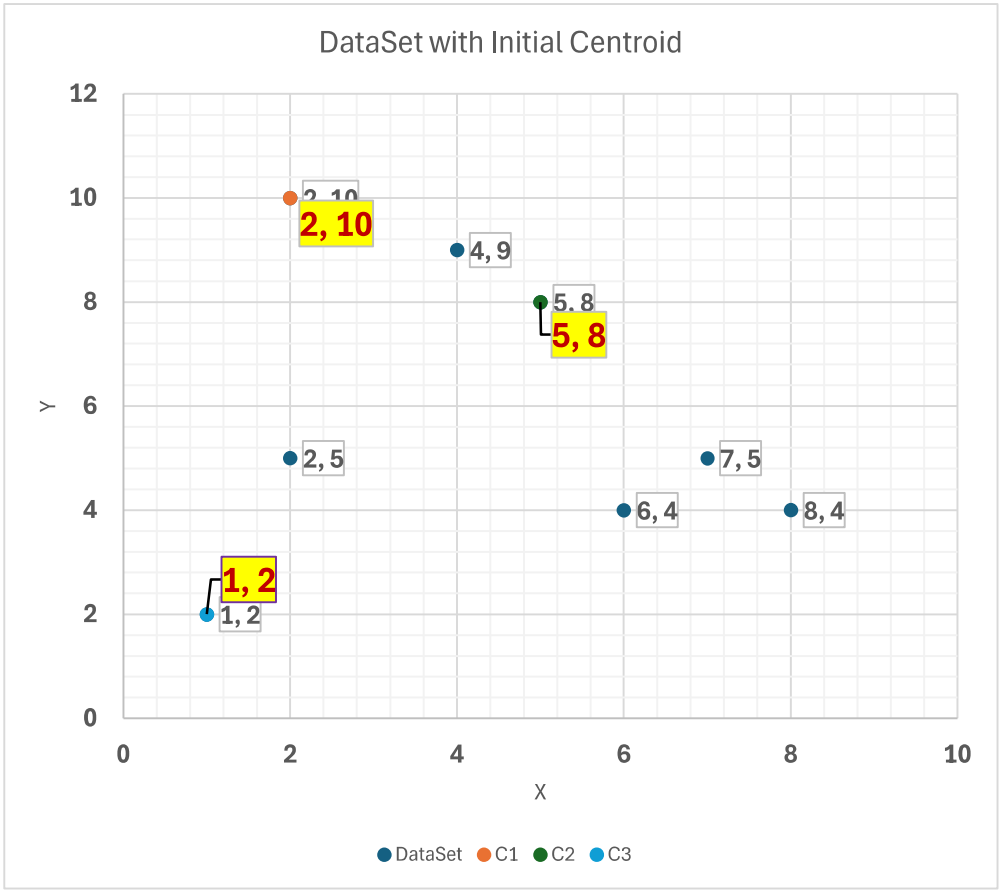
# K-Mean: Calculation – Iteration-One

| Data Set | | C1 | C2 | C3 | Similarity(C1,C2,C3) | Cluster | DataSet | | Average(X) | Average(Y) |
|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 10 | 0.00 | 3.61 | 8.06 | 0.00 | C1 | 2 | 10 | 2 | 10 |
| 2 | 5 | 5.00 | 4.24 | 3.16 | 3.16 | C3 | 2 | 5 | | |
| 8 | 4 | 8.49 | 5.00 | 7.28 | 5.00 | C2 | 8 | 4 | | |
| 5 | 8 | 3.61 | 0.00 | 7.21 | 0.00 | C2 | 5 | 8 | 6 | 6 |
| 7 | 5 | 7.07 | 3.61 | 6.71 | 3.61 | C2 | 7 | 5 | | |
| 6 | 4 | 7.21 | 4.12 | 5.39 | 4.12 | C2 | 6 | 4 | | |
| 1 | 2 | 8.06 | 7.21 | 0.00 | 0.00 | C3 | 1 | 2 | 1.5 | 3.5 |
| 4 | 9 | 2.24 | 1.41 | 7.62 | 1.41 | C2 | 4 | 9 | | |

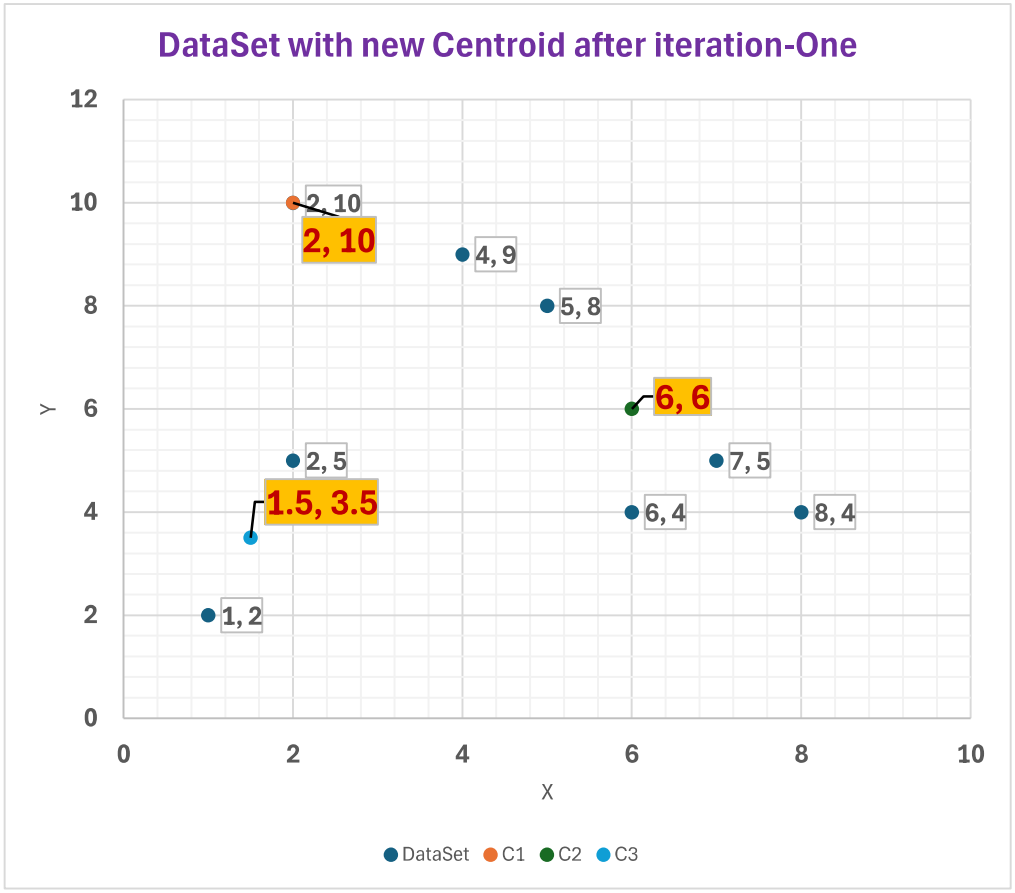
| | | |
|---|---|---|
| C1 | 2 | 10 |
| C2 | 5 | 8 |
| C3 | 1 | 2 |

| | | |
|---|---|---|
| C1 (Avg) | 2 | 10 |
| C2(Avg) | 6 | 6 |
| C3(Avg) | 1.5 | 3.5 |

# K-Mean: Centroid Shifting after Iteration-One




Centroid before Iteration-One




Centroid after Iteration-One

# K-Mean: Centroid Shifting after Iteration-Two

| | | C1 | C2 | C3 | Similarity(C1,C2,C3) | Old | New | | | Average(X) | Average(Y) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 10 | 0.00 | 5.66 | 6.52 | 0.00 | C1 | C1 | 2 | 10 | 3.5 | 9 |
| 2 | 5 | 5.00 | 4.12 | 1.58 | 1.58 | C3 | C3 | 2 | 5 | | |
| 8 | 4 | 8.49 | 2.83 | 6.52 | 2.83 | C2 | C2 | 8 | 4 | | |
| 5 | 8 | 3.61 | 2.24 | 5.70 | 2.24 | C2 | C1 | 5 | 8 | 6.5 | 5.25 |
| 7 | 5 | 7.07 | 1.41 | 5.70 | 1.41 | C2 | C2 | 7 | 5 | | |
| 6 | 4 | 7.21 | 2.00 | 4.53 | 2.00 | C2 | C2 | 6 | 4 | | |
| 1 | 2 | 8.06 | 6.40 | 1.58 | 1.58 | C3 | C3 | 1 | 2 | 1.5 | 3.5 |
| 4 | 9 | 2.24 | 3.61 | 6.04 | 2.24 | C2 | C2 | 4 | 9 | | |

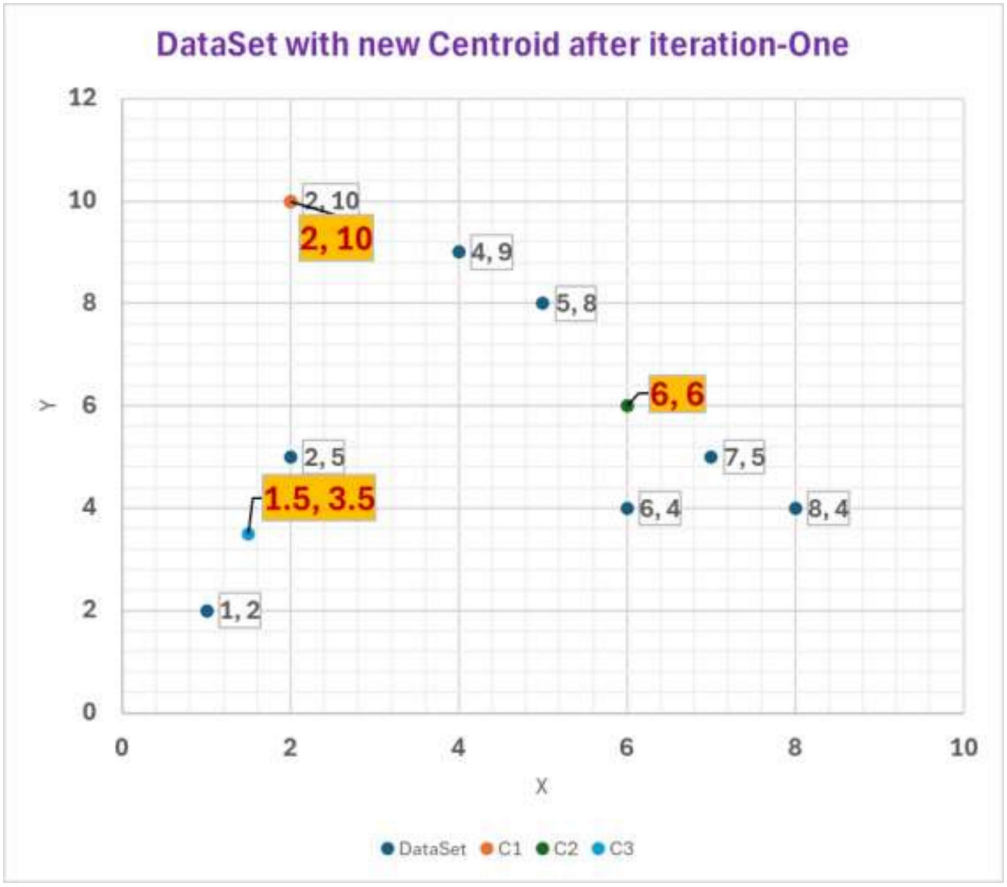
| | | |
|---|---|---|
| C1 | 2 | 10 |
| C2 | 6 | 6 |
| C3 | 1.5 | 3.5 |

Old Centroid

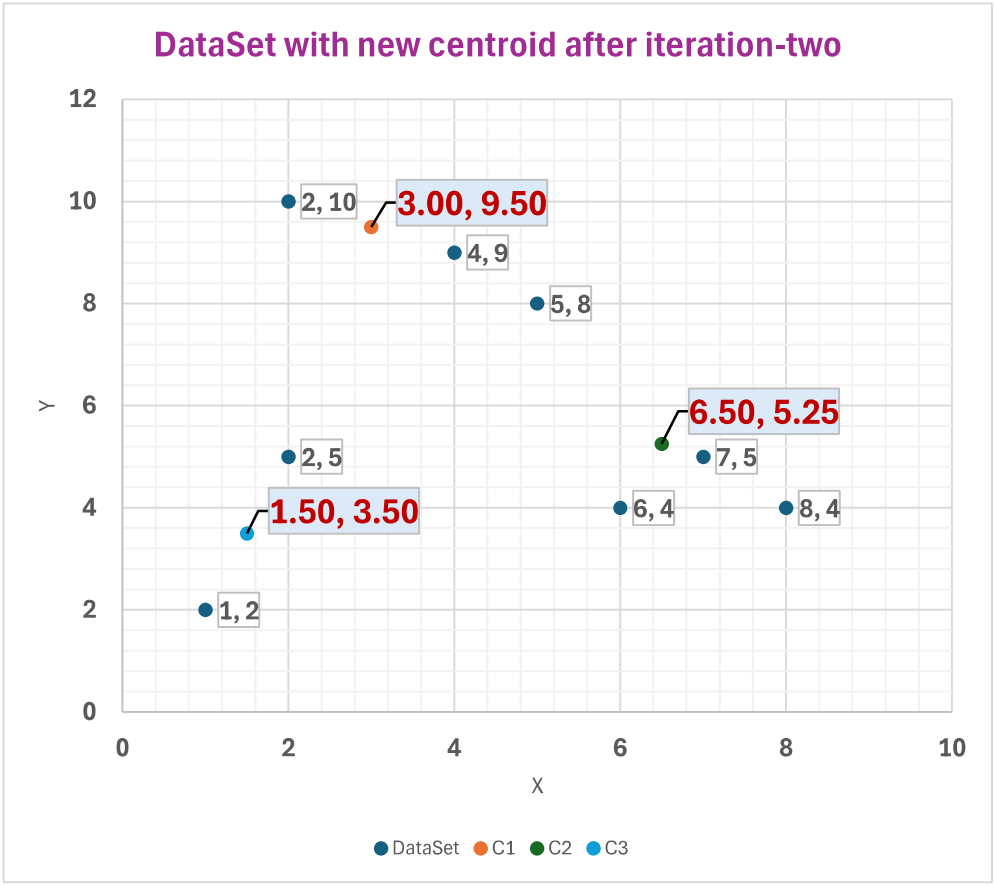
| | | |
|---|---|---|
| C1 (Avg) | 3.00 | 9.50 |
| C2(Avg) | 6.50 | 5.25 |
| C3(Avg) | 1.50 | 3.50 |

New Centroid

# K-Mean: Centroid Shifting after Iteration-Two




Centroid before Iteration-Two




Centroid after Iteration-Two

# K-Mean: Centroid Shifting after Iteration-Three

| | | C1 | C2 | C3 | Similarity(C1,C2,C3) | | | | | Average(X) | Average(Y) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 10 | 1.12 | 6.54 | 6.52 | 1.12 | C1 | C1 | 2 | 10 | 3.67 | 9.00 |
| 2 | 5 | 4.61 | 4.51 | 1.58 | 1.58 | C3 | C3 | 2 | 5 | | |
| 8 | 4 | 7.43 | 1.95 | 6.52 | 1.95 | C2 | C2 | 8 | 4 | | |
| 5 | 8 | 2.50 | 3.13 | 5.70 | 2.50 | C2 | C1 | 5 | 8 | 7.00 | 4.33 |
| 7 | 5 | 6.02 | 0.56 | 5.70 | 0.56 | C2 | C2 | 7 | 5 | | |
| 6 | 4 | 6.26 | 1.35 | 4.53 | 1.35 | C2 | C2 | 6 | 4 | | |
| 1 | 2 | 7.76 | 6.39 | 1.58 | 1.58 | C3 | C3 | 1 | 2 | 1.50 | 3.50 |
| 4 | 9 | 1.12 | 4.51 | 6.04 | 1.12 | C2 | C1 | 4 | 9 | | |

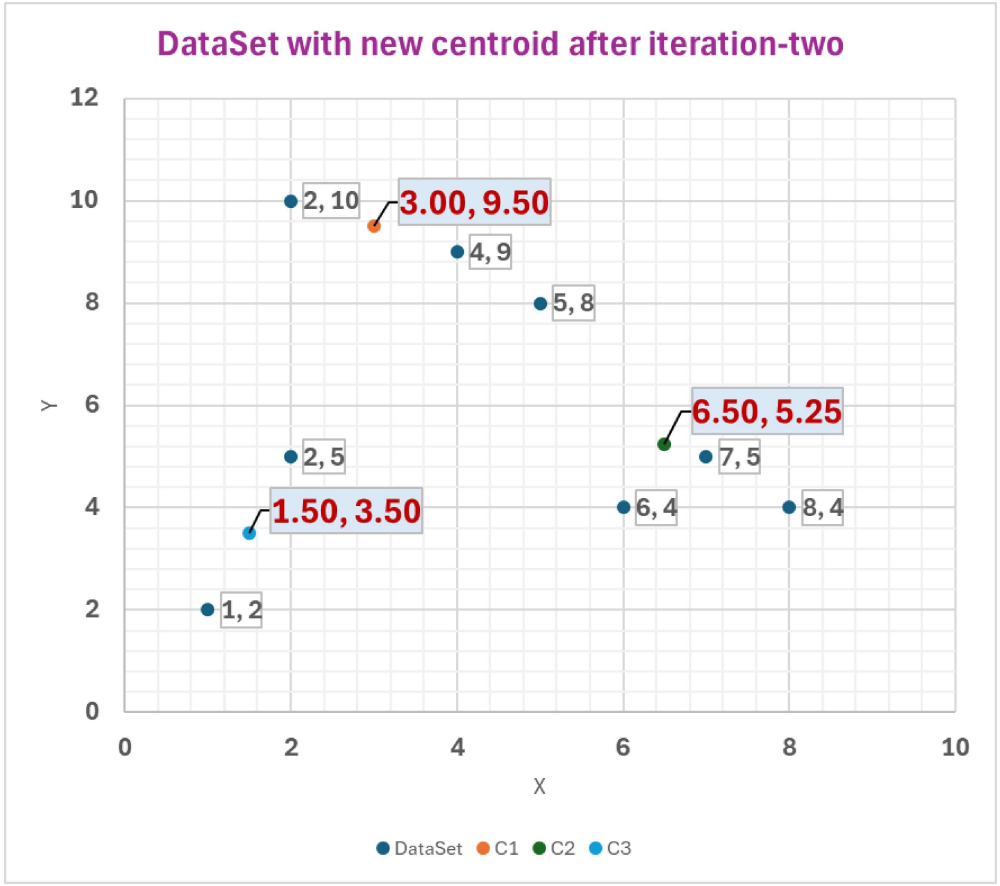
| | | |
|---|---|---|
| C1 | 3.00 | 9.50 |
| C2 | 6.50 | 5.25 |
| C3 | 1.50 | 3.50 |

Old Centroid

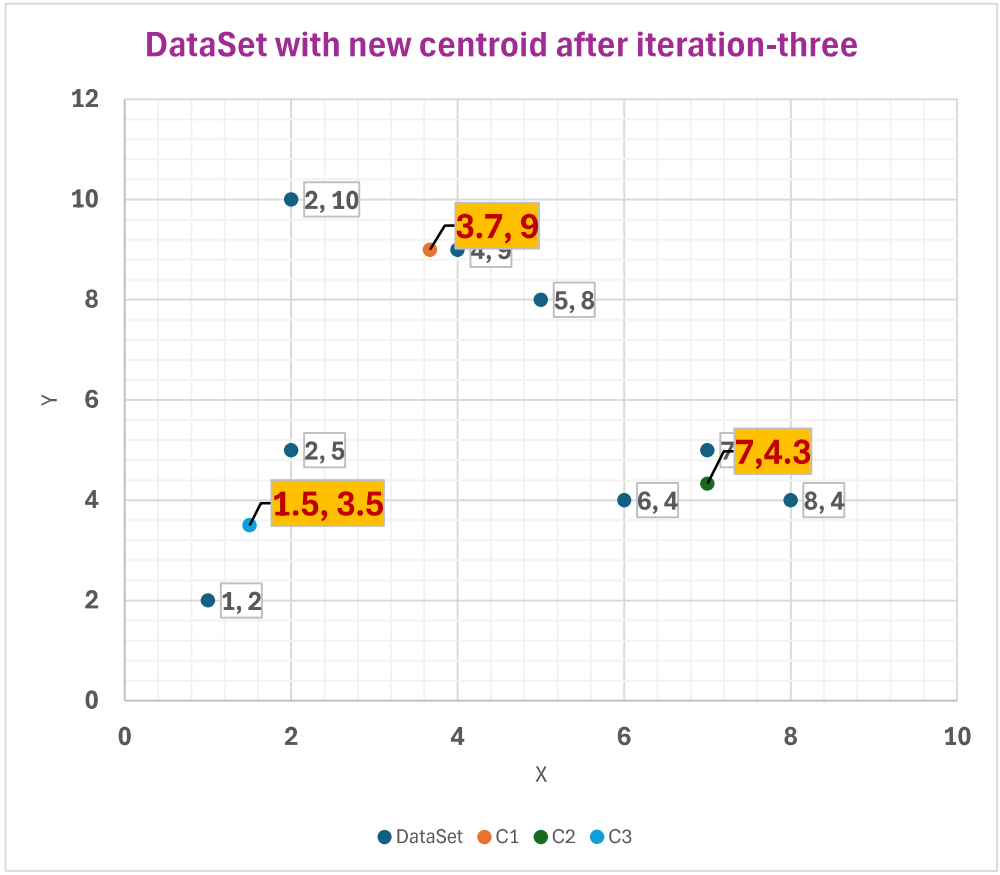
| | | |
|---|---|---|
| C1 (Avg) | 3.67 | 9 |
| C2(Avg) | 7 | 4.33 |
| C3(Avg) | 1.5 | 3.5 |

New Centroid

# K-Mean: Centroid Shifting after Iteration-Three




Centroid before Iteration-Three




Centroid after Iteration-Three

# K-Mean: Centroid Shifting after Iteration-Four

| | | C1 | C2 | C3 | Similarity(C1,C2,C3) | | | | | Average(X) | Average(Y) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 10 | 1.94 | 7.56 | 6.52 | 1.94 | C1 | C1 | 2 | 10 | 3.67 | 9.00 |
| 2 | 5 | 4.33 | 5.04 | 1.58 | 1.58 | C3 | C3 | 2 | 5 | | |
| 8 | 4 | 6.62 | 1.05 | 6.52 | 1.05 | C2 | C2 | 8 | 4 | | |
| 5 | 8 | 1.67 | 4.18 | 5.70 | 1.67 | C1 | C1 | 5 | 8 | 7.00 | 4.33 |
| 7 | 5 | 5.21 | 0.67 | 5.70 | 0.67 | C2 | C2 | 7 | 5 | | |
| 6 | 4 | 5.52 | 1.05 | 4.53 | 1.05 | C2 | C2 | 6 | 4 | | |
| 1 | 2 | 7.49 | 6.44 | 1.58 | 1.58 | C3 | C3 | 1 | 2 | 1.50 | 3.50 |
| 4 | 9 | 0.33 | 5.55 | 6.04 | 0.33 | C1 | C1 | 4 | 9 | | |

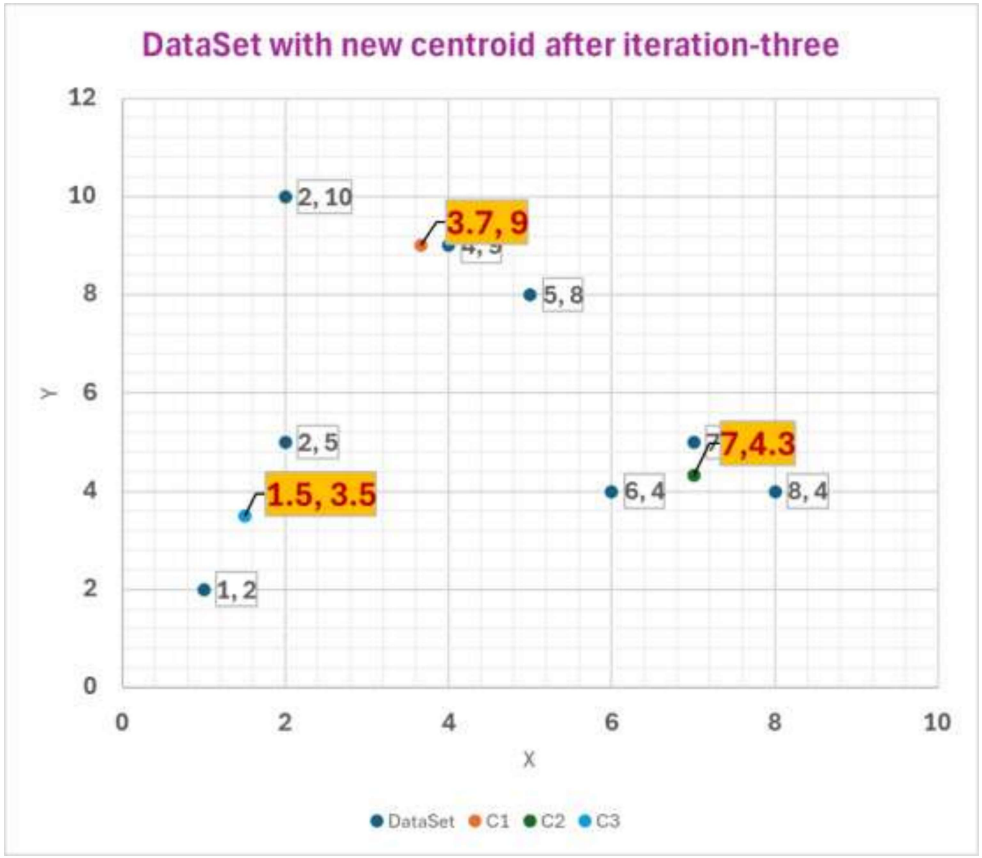
| | | |
|---|---|---|
| C1 | 3.67 | 9 |
| C2 | 7 | 4.33 |
| C3 | 1.5 | 3.5 |

Old Centroid

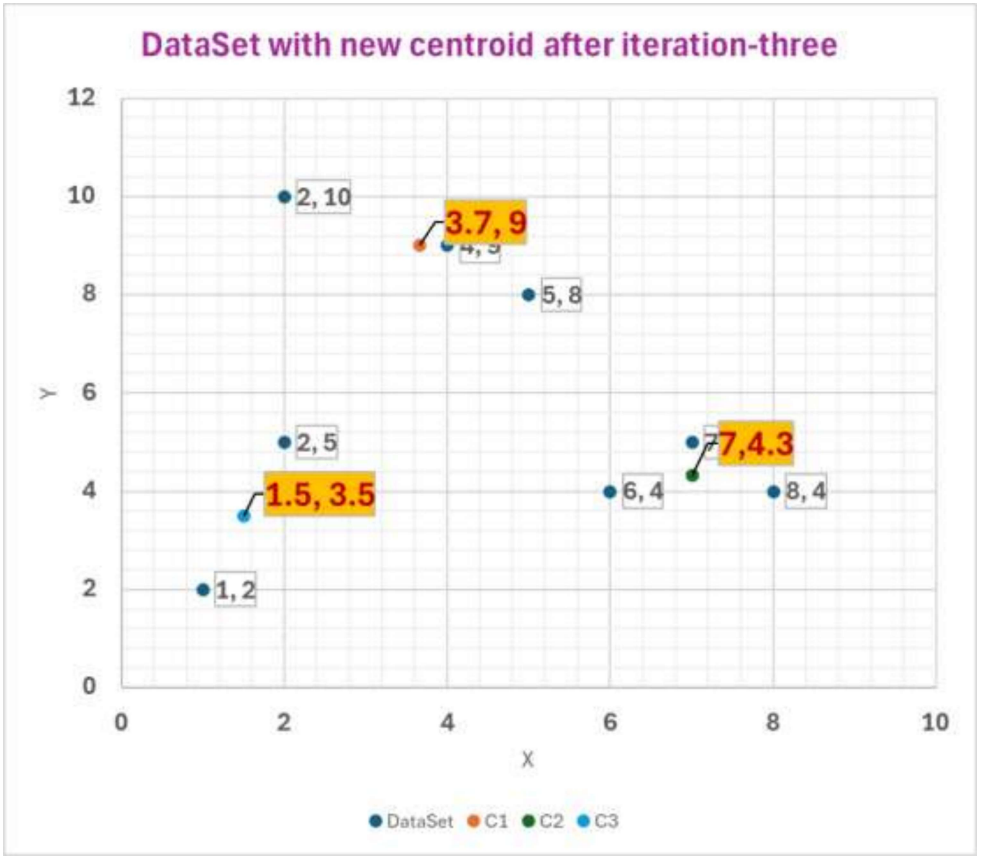
| | | |
|---|---|---|
| C1 (Avg) | 3.67 | 9 |
| C2(Avg) | 7 | 4.33 |
| C3(Avg) | 1.5 | 3.5 |

New Centroid

# K-Mean: Centroid Shifting after Iteration-Four



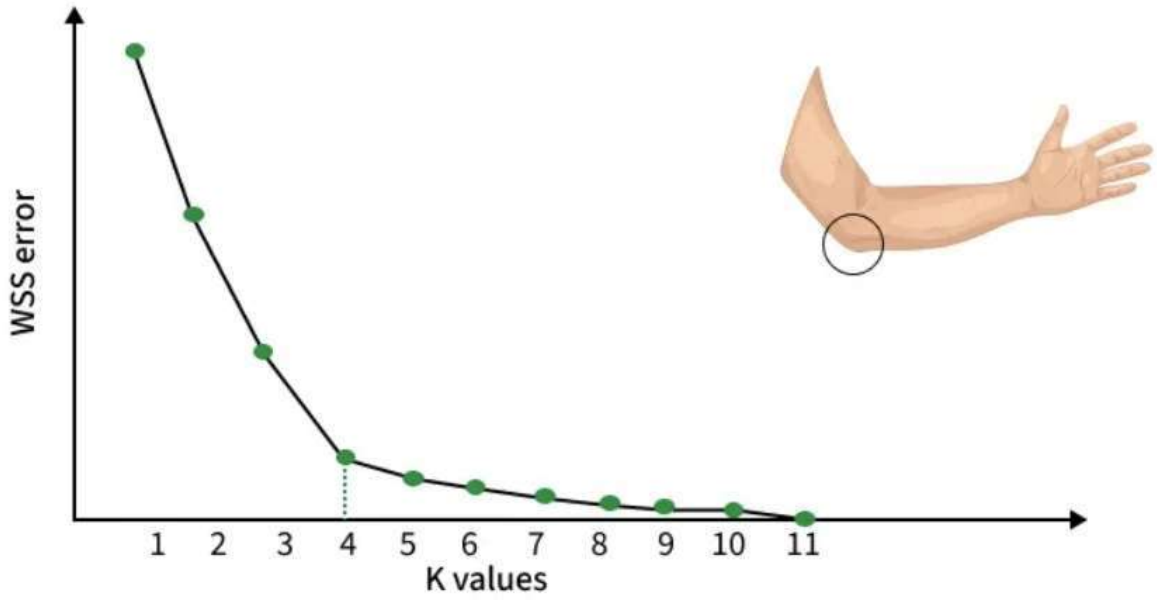

Centroid before Iteration-four




Centroid after Iteration-four

## Elbow Method for optimal value of k in KMeans

The Elbow Method helps by plotting the Within-Cluster Sum of Squares (WCSS) against increasing k values and looking for a point where the improvement slows down, this point is called the "elbow."



## Elbow Method for optimal value of k in KMeans

We begin by selecting a range of k values (for example, 1 to 10).

For each k, we run K-Means and calculate WCSS (Within-Cluster Sum of Squares), which shows how close the data points are to their cluster centroids:

$$\text{WCSS} = \sum_{i=1}^{k} \sum_{j=1}^{n_i} \text{distance}\left(x_j^{(i)}, c_i\right)^2$$

Where $\text{distance}\left(x_j^{(i)}, c_i\right)$ represents the distance between the $j^{th}$ data point $x_j^{(i)}$ in cluster i and the centroid $c_i$ of that cluster.

After computing WCSS for all k values, we plot k vs WCSS.

WCSS always decreases as k increases because more clusters reduce the internal spread.

However, after a certain point, the improvement becomes very small. This bend or "elbow" in the curve indicates the point where adding more clusters no longer gives meaningful improvement.